services\data\data_processor.py

```
import os
import json
import gzip
import gc
from typing import List, Dict, Any, Optional
from datetime import datetime
import config
from database.db_manager import db_manager


class HistoricalDataProcessor:
"""
Loads historical CVEs year-by-year in a memory-safe way.
DB-first: if DB has the year's records, we use those.
If DB lacks them and local files exist, we parse files (locally),
then (optionally) store to DB.
"""

def __init__(self, historical_dir: Optional[str] = None):
self.historical_dir = historical_dir or config.NVD_HISTORICAL_DIR
# Only keep ONE year's worth of parsed JSON in RAM
self._year_cache: Dict[str, List[Dict[str, Any]]] = {}
self._max_cache_size = 1
self._years_available = set()
print(f"[Historical] DataProcessor initialized")
print(f"[Historical] Local directory: {self.historical_dir}")
self._check_available_files()

# --------- file discovery helpers ----------
def _check_available_files(self):
"""Find available historical files (for local/dev)."""
self._years_available.clear()
if not os.path.exists(self.historical_dir):
print(f"[Historical] Directory does not exist: {self.historical_dir}")
try:
os.makedirs(self.historical_dir, exist_ok=True)
except Exception:
pass
return

files = os.listdir(self.historical_dir)
cve_files = [f for f in files if f.startswith(('CVE-', 'nvdcve-'))]
if cve_files:
print(f"[Historical] Found {len(cve_files)} files: {cve_files}")
for file in cve_files:
year = None
if file.startswith("CVE-") and "." in file:
year_str = file[4:file.find(".")]
if year_str.isdigit() and len(year_str) == 4:
year = int(year_str)
elif file.startswith("nvdcve-") and "-" in file:
parts = file.split("-")
if len(parts) >= 3 and parts[2].isdigit() and len(parts[2]) == 4:
year = int(parts[2])
if year and 2000 <= year <= 2050:
self._years_available.add(year)

if self._years_available:
print(f"[Historical] Available years: {sorted(self._years_available)}")
else:
```

```
print(f"[Historical] WARNING: No files found in {self.historical_dir}")

def get_available_years(self) -> List[int]:
"""Return the list of available local years (for local/dev)."""
if not self._years_available:
self._check_available_files()
return sorted(self._years_available, reverse=True)

# --------- main data access ----------
def get_year_data(self, year: int) -> List[Dict[str, Any]]:
"""
Get CVEs for a specific year.
- Prefer DB (Render).
- Fall back to memory cache.
- Finally, load from local file if allowed/available (local/dev).
"""
year_str = str(year)

# 1) Prefer DB
if db_manager.use_database:
cves = db_manager.get_cves_by_filter(year=year_str)
if cves:
print(f"[Historical] Retrieved {len(cves)} CVEs for {year} from database")
return cves

# 2) Use in-memory year cache
if year_str in self._year_cache:
print(f"[Historical] Using cached data for {year}")
return self._year_cache[year_str]

# 3) Try to load from local feed if enabled (local/dev)
if config.USE_LOCAL_FEEDS:
data = self._load_year_file(year)
if db_manager.use_database and data:
print(f"[Historical] Saving {len(data)} CVEs for {year} to database")
db_manager.save_cves_batch(data)
# keep just one year in memory
if len(self._year_cache) >= self._max_cache_size:
oldest = next(iter(self._year_cache.keys()))
print(f"[Historical] Evicting {oldest} from cache")
del self._year_cache[oldest]
gc.collect()
self._year_cache[year_str] = data
return data

# If local feeds disabled and DB empty, return empty list (safe default)
print(f"[Historical] No DB data for {year} and local feeds disabled")
return []

# ---------- local file parsing (supports .json or .json.gz) ----------
def _load_year_file(self, year: int) -> List[Dict[str, Any]]:
"""
Load a year file from historical_dir (JSON or GZ) in a streaming-safe way.
Supports multiple naming patterns (nvdcve-1.1-YYYY.json, etc.).
"""
patterns = [
f"CVE-{year}.json",
f"nvdcve-1.1-{year}.json",
f"nvdcve-2.0-{year}.json",
f"CVE-{year}.json.gz",
f"nvdcve-1.1-{year}.json.gz",
f"nvdcve-2.0-{year}.json.gz",
]
filepath = None
```

```
for name in patterns:
candidate = os.path.join(self.historical_dir, name)
if os.path.exists(candidate):
filepath = candidate
break

if not filepath:
print(f"[Historical] No historical file found for {year}")
return []

try:
open_f = gzip.open if filepath.endswith(".gz") else open
with open_f(filepath, "rt", encoding="utf-8", errors="ignore") as f:
data = json.load(f)
except Exception as e:
print(f"[Historical] Error reading {filepath}: {e}")
return []

# NVD JSON 2.0 ? "vulnerabilities": [ { "cve": {...}}, ... ]
vulns = data.get("vulnerabilities") or data.get("CVE_Items") or []
out: List[Dict[str, Any]] = []
for item in vulns:
cve_obj = item.get("cve", item)  # support both shapes
processed = self._process_cve_item(cve_obj)
if processed:
out.append(processed)

print(f"[Historical] Loaded {len(out)} CVEs from
{os.path.basename(filepath)}")
return out

# ---------- normalize NVD CVE into your app schema ----------
def _process_cve_item(self, cve_data: Dict[str, Any]) ->
Optional[Dict[str, Any]]:
try:
cve_id = cve_data.get("id") or cve_data.get("CVE_data_meta", {}).get("ID")
if not cve_id:
return None

# Description
description = ""
desc = cve_data.get("descriptions") or cve_data.get("description",
{}).get("description_data")
if isinstance(desc, list) and desc:
description = desc[0].get("value") or ""
elif isinstance(desc, str):
description = desc

# Severity / CVSS
severity = "UNKNOWN"
cvss_score = None
metrics = cve_data.get("metrics") or {}
for key in ["cvssMetricV31", "cvssMetricV30", "cvssMetricV2",
"cvssMetricV3"]:
arr = metrics.get(key)
if isinstance(arr, list) and arr:
m = arr[0].get("cvssData") or arr[0].get("baseMetricV3", {}).get("cvssV3",
{})
if not m and isinstance(arr[0], dict):
m = arr[0].get("cvssData", {})
cvss_score = m.get("baseScore")
sev = m.get("baseSeverity") or arr[0].get("baseSeverity")
if isinstance(sev, str):
severity = sev.upper()
```

```
break

# CWE (collect full list if available)
cwe = "Unknown"
cwe_list = []
weaknesses = cve_data.get("weaknesses") or []
for w in weaknesses:
descs = w.get("description", [])
for d in descs:
val = d.get("value")
if val and val.startswith("CWE-"):
cwe_list.append(val)
if cwe_list:
cwe = cwe_list[0]

# Dates
published = (
cve_data.get("published")
or cve_data.get("publishedDate")
or cve_data.get("datePublished")
or ""
)
last_modified = (
cve_data.get("lastModified")
or cve_data.get("lastModifiedDate")
or ""
)

# ? Normalize date format for charts
published_str = ""
if published:
try:
dt = datetime.fromisoformat(published.replace("Z", "+00:00"))
published_str = dt.strftime("%Y-%m-%d")
except Exception:
published_str = str(published).split("T")[0]

# References
refs = []
references = cve_data.get("references") or cve_data.get("reference_data",
[])
if isinstance(references, list):
for r in references:
url = r.get("url") or r.get("refsource")
if url:
refs.append(url)

# ? FINAL normalized return
return {
"ID": cve_id,
"Description": description or "No description available",
"Severity": severity,
"CVSS_Score": cvss_score,
"CWE": cwe,
"cwe_id": cwe_list,
"Published": published_str,
"published_date": published_str,  # fixed format YYYY-MM-DD
"lastModified": last_modified or "",
"References": refs,
"Products": [],
"metrics": {},
}

except Exception as e:
```

```
print(f"[Historical] Error processing CVE item: {e}")
return None


# Global instance used by analyzers
historical_loader = HistoricalDataProcessor()
```